

Rapport individuel

Périmètre observabilité et sécurité

Auteur

Nicolas Roulois

Client

Korlea

Mastère DevOps M1, SUP DE VINCI Paris La Défense

Promotion M1DOA 2025-2026

1. Pourquoi ce document, et ce que je porte vraiment

Le DAT groupe décrit toute la plateforme et m'attribue l'observabilité et la sécurité en §1.2.3. Plutôt que de répéter ce qu'il dit, ce rapport revient sur ma partie en détail : ce que j'ai écrit et calibré, ce que j'ai seulement branché, et les limites que j'ai documentées. Pour les faits, il s'appuie sur le DAT, en particulier §4.7 et §4.8.

Avant d'entrer dans le détail, une mise au point utile. Le DAT attribue les briques en bloc, mais dans les faits ma contribution se situe à trois niveaux différents, que je préfère distinguer.

Ce que je porte de bout en bout : les quatre NetworkPolicies applicatives, le calibrage du SLO multi-burn-rate en conditions réelles, le contenu des dashboards, la structuration de l'investigation tri-canal. Ce que je remplis sur une structure posée par Tom : le contenu des *PrometheusRule* et des *ConfigMap* de dashboard, dont il a écrit le squelette tôt dans le projet. Ce que je branche sans l'avoir monté : la chaîne Vault, Dex et External-Secrets, où j'écris les *ExternalSecret* côté application et je documente les limites, mais où je ne touche ni au bootstrap ni au sync wave.

Cette grille tient tout le rapport. Je revendique le premier niveau sans réserve. Je ne m'attribue pas le travail de Tom sur les deux autres.

2. Perspectives d'évolution

2.1 Vault en mode production

La limite la plus critique de ma partie sécurité tient au mode de Vault. Le serveur tourne en mode DEV (chart 0.31.0, *kubernetes/applications/security/vault.yaml*), avec un root token statique en clair dans le manifest (ligne 25). Ce mode évite l'unseal manuel et laisse External-Secrets se connecter sans préparation, pratique en démo mais pas en production.

Le passage en production remplacerait ce mode par une configuration *standalone* avec stockage persistant, ou *ha* avec etcd embarqué Raft si trois instances sont déployables. Le scellement s'appuierait sur un KMS externe (AWS KMS, GCP KMS, ou un HSM pour un déploiement on-premise sensible), les politiques deviendraient fines par chemin de secret, et le root token serait supprimé après bootstrap au profit de tokens applicatifs scoped.

Je nomme ce chantier en premier parce qu'il est le plus exposé sur ma partie. Je précise aussi sa frontière : le montage de Vault et son bootstrap relèvent de Tom. Mon angle reste le branchement applicatif, les trois *ExternalSecret* du namespace, et la documentation des limites. La condition de production que je porte est celle de la couche que je touche : des secrets réellement sourcés de Vault avec une rotation effective, pas un placeholder.

2.2 Restaurer OIDC réel

L'application traite aujourd'hui toute requête comme **operator**. Les variables OIDC sont commentées dans *kubernetes/apps/projet-etude-app-demo/deployment-api.yaml:83-104*, et le client secret Dex reste un placeholder (*dex.yaml:45*). Ce choix simplifie la démo. Il vide aussi l'audit log d'une partie de son sens : sans identité réelle, la traçabilité que j'exploite côté observabilité enregistre un seul utilisateur fictif.

Restaurer OIDC consisterait à décommenter ces variables, brancher un connecteur Dex upstream (GitHub, LDAP ou SAML selon le client), et rotater le client secret en le sourçant depuis Vault via un *ExternalSecret*. C'est le chantier qui redonne sa valeur à l'audit log, donc à toute la chaîne de traçabilité RGPD au runtime.

2.3 Fermer le trou egress du webhook

La NetworkPolicy `worker` autorise l'egress vers le webhook externe sur le port 443, ouvert à toute destination (*networkpolicy.yaml:113*). C'est un trou que j'ai laissé volontairement, et documenté en commentaire dans le manifest. Je voulais filtrer par FQDN. Le CNI par défaut de K3s ne sait pas le faire, et déployer Cilium uniquement pour cette règle ne se justifiait pas dans le périmètre démo. Je l'ai évaluée avant de l'écarter.

En production, une *FQDN-aware policy* (Cilium ou équivalent) restreindrait cette destination à l'URL réelle du webhook. La condition n'est pas qu'une ligne de YAML : elle suppose un changement de CNI, donc un arbitrage qui dépasse ma seule NetworkPolicy.

2.4 Hors DAT, deux pistes qui m'intéressent

La rétention d'observabilité est courte : sept jours sur Prometheus, Loki et Tempo. Ce choix tient pour les investigations post-incident, qui portent presque toujours sur la semaine écoulée. Il interdit les tendances longues. Une base remote-write longue durée (Mimir ou Thanos) conserverait les recording rules au-delà de sept jours et ouvrirait les comparaisons mensuelles et annuelles.

Le reporting énergétique manque d'un écran dédié. Les métriques Kepler entrent dans Prometheus (`kepler_container_joules_total` par conteneur), mais aucun dashboard direction ne les expose. Un panel des joules par requête et par job traité rendrait le Green IT lisible pour une direction, pas seulement calculable en PromQL par un opérateur. Les deux pistes se complètent : sans rétention longue, pas de tendance énergétique sur un trimestre.

3. Analyse critique des limites techniques rencontrées

3.1 La limite la plus chronophage : Kepler en VM

Le problème qui m'a fait perdre le plus de temps sur tout le projet, c'est Kepler qui refusait de produire des métriques correctes sur les VMs Proxmox, sur deux ou trois soirées de mars et avril 2026. Les logs répétaient `RAPL unavailable`, et je suis parti dans la mauvaise direction : permissions kernel, configuration eBPF, accès bas niveau au matériel. Je croyais avoir une panne à réparer.

Tom m'a débloqué en une phrase : regarde `MODEL_CONFIG`. Une demi-heure plus tard, c'était réglé. J'ai trouvé `CONTAINER_COMPONENTS_ESTIMATOR=true` dans *kubernetes/applications/observability/kepler.yaml*, et j'ai compris ce que le log essayait de me dire. Kepler lit normalement les compteurs Intel RAPL pour mesurer la consommation au niveau matériel. Dans une VM Proxmox, ce compteur n'existe pas : la VM n'a pas d'accès direct au PMC de l'hôte. Kepler bascule alors sur un modèle d'estimation par apprentissage machine embarqué. Les métriques étaient là dès le début, mais estimées plutôt que mesurées.

Avec le recul, ce message décrivait surtout la nature de mon environnement de test.

Presque tout le temps perdu était dans l'interprétation, pas dans la correction. Le fix tenait dans deux lignes de configuration que je n'avais pas lues pour ce qu'elles disaient. Ce que je retiens, c'est moins de lire les logs, je les lisais déjà, que de distinguer un message qui signale un défaut d'un message qui décrit un environnement. La précision moindre de Kepler en VM, un ordre de grandeur exploitable entre pods mais une valeur absolue non calibrée, est une limite que j'ai documentée ensuite plutôt que de la traiter comme un bug.

3.2 Les compromis assumés sur le périmètre sécurité

Plusieurs compromis pèsent sur ma partie, et aucun n'est vraiment une erreur : ce sont des décisions prises pour la démo, qui se voient surtout sur la sécurité.

Le mode Vault DEV est un choix collectif. Il fait gagner du temps à toute l'équipe en supprimant l'unseal manuel à chaque redémarrage du pod. Mais le root token en clair dans le manifest est une faille, et elle tombe dans mon périmètre. Je l'ai documentée dans le manifest et dans le DAT au lieu de la laisser passer en silence.

L'OIDC désactivé est un compromis que je subis plus que je ne choisis. Mon rôle côté traçabilité repose sur l'audit log. Tant que toute requête passe en `operator`, cet audit log enregistre une fiction. Je vis avec, et je sais exactement ce que ça coûte.

Le trou egress du webhook complète la liste, je l'ai décrit plus haut. Le fil commun de ces trois compromis tient à une règle que je me suis tenue : nommer la limite plutôt que la masquer, pour pouvoir la défendre au lieu de la découvrir trop tard.

3.3 Ce que je ferais différemment

Si je reprenais ce périmètre aujourd'hui, je changerais trois choses.

Je mettrais OIDC en route dès le début, même avec un Dex minimal et un seul connecteur. L'audit log aurait une vraie valeur dès la première itération, au lieu d'attendre une restauration qui n'a pas eu lieu dans le temps du projet.

Je documenterais les queries PromQL dans un fichier dédié dès la première version des dashboards. Aujourd'hui elles sont enfouies dans le JSON des *ConfigMap*. C'est le point où je suis le plus fragile, je les adapte sans toutes les inventer, et les laisser illisibles ne m'aide pas à les défendre.

Je testerais les alertes SLO avec les recettes `just demo-flaky` et `just demo-slow` dès le premier dashboard, pas à la fin. J'ai calibré juste, mais tard. Tester tôt aurait transformé une série de soirées de mise au point en une boucle courte intégrée au développement.

4. Annexes

4.1 Documentation utilisateur : lire un incident dans Grafana

Cette annexe documente le parcours que j'ai le plus répété pendant les démos internes : lire un incident de bout en bout dans Grafana, sans ouvrir un shell. Elle s'adresse à un opérateur qui reçoit une alerte et veut remonter à la cause en quelques clics. C'est le mode d'emploi de la corrélation tri-canal vue depuis le siège de l'astreinte, pas depuis l'architecture.

Le point de départ est l'alerte. Une alerte burn-rate arrive sur le salon Discord `#alerts`. Deux niveaux à distinguer tout de suite : `AppDemoSLOErrorBudgetBurnFast` (critique, seuil 14.4x sur 5 minutes et 1 heure) appelle une réaction immédiate, `AppDemoSLOErrorBudgetBurnSlow` (warning, seuil 6x sur 30 minutes et 6 heures) signale une dérive lente.

1. Ouvrir le dashboard **SLO et burn-rate** (UID `projet-etude-app-demo-slo`). Les quatre fenêtres (5m, 30m, 1h, 6h) sont sur un même graphe. Repérer celle qui décroche et le moment du pic.
2. Sur le graphe d'erreurs ou de latence, cliquer sur l'exemplar du bucket le plus dégradé. L'exemplar est le marqueur en losange ; il porte le `trace_id` du span actif au moment de la mesure.

3. Le clic ouvre la trace dans Tempo. Lire la chaîne de spans : quel service, dans quel ordre, quelle étape a pris le temps ou a échoué.
4. Dans la trace, cliquer sur **Logs for this span**. Grafana ouvre Loki filtré sur le `trace_id`. On lit les logs structurés `slog` du seul pod concerné, sans grep manuel.
5. Le message d'erreur en main, souvent une erreur Postgres ou un timeout, remonter au code via l'historique Git.

L'enchaînement tient sous la minute une fois qu'on a le geste. Le point que je répète à qui apprend le dashboard : ne pas commencer par les logs. On part de l'alerte, on descend par la trace, on finit dans les logs. L'inverse fait perdre du temps à chercher dans tout ce que le cluster a produit.

4.2 Analyse personnelle

Défis rencontrés

Le défi qui m'a le plus appris n'était pas un bug mais le calibrage du SLO en conditions réelles. Le seuil de 14.4x vient du Google SRE Workbook, et la structure des règles Prometheus était posée tôt, mais recopier un seuil ne suffit pas. Pendant les démos internes, j'ai lancé `just demo-flaky` et `just demo-slow` plusieurs fois pour voir comment les alertes réagissaient au trafic réel. Une fois, le burn-rate a sauté sur un spike isolé qui ne le méritait pas. J'ai ajusté la fenêtre, retesté, et vérifié que la double fenêtre 5m plus 1h tenait sans faux positif. Ce calibrage, je l'ai donc éprouvé en conditions réelles, pas seulement sur le papier.

L'autre moment qui compte est plus simple. La première fois que j'ai vu une trace continue, de la requête HTTP jusqu'au worker, passer par la colonne `trace_context` JSONB de la table `jobs`, tout ce que l'observabilité tri-canal promet en théorie est devenu concret.

Forces et faiblesses

Mes points forts sont clairs. J'écris et je comprends les NetworkPolicies ligne par ligne, je calibre l'alerting contre le trafic réel plutôt que sur des seuils recopiés, et je structure l'investigation tri-canal pour que les autres déboguent sans entrer dans un conteneur. Ce point compte plus qu'il n'en a l'air : les images sont en distroless, sans shell, donc le jour où Karim ne peut plus faire `kubectl exec`, c'est ma stack et ses traces qui prennent le relais.

J'ai aussi des limites assez claires. Sur la couche Vault et secrets, je la branche et je documente les flux, mais si Vault ne démarre pas, je ne sais pas le réparer seul : je dépends de Tom pour le bootstrap, et je l'assume. Autre point que je préfère reconnaître, les queries PromQL complexes, les quantiles sur histograms et les `rate()` sur counters, je les adapte sans toutes les inventer. C'est pour ça que je veux les documenter à part, parce que ce que je n'écris pas seul, je dois au moins savoir l'expliquer.

Compétences développées

Avant le projet, je connaissais Prometheus en cours (scrape, counters, gauges) et Grafana au niveau du panel simple et de l'import de dashboard. Zéro OpenTelemetry, zéro Tempo, zéro Loki, zéro networking Kubernetes. À la sortie, je tiens une stack CNCF complète : SLO multi-burn-rate, exemplars Prometheus, derived fields Loki vers Tempo, propagation `trace_id` cross-process, NetworkPolicies deny-by-default, et le fallback ML de Kepler que j'ai compris à la dure.

Sur l'usage de l'IA, je veux être précis plutôt que vague. J'ai utilisé Claude Code comme accélérateur de compréhension : générer les structures YAML des *ConfigMap* de dashboard, déboguer des erreurs Alloy quand les logs ne portaient pas vers Loki, me faire expliquer le

format des exemplars Prometheus. Je m'en suis aussi servi pour la rédaction, en m'appuyant dessus pour structurer et relire le DAT comme ce rapport. Je ne l'ai jamais utilisé pour les queries PromQL critiques des alertes, par peur d'un faux positif déclenchant une astreinte. Sur ces queries, je valide à la main. L'IA m'a aidé à comprendre plus vite, sans remplacer la validation.

Axes d'amélioration pour de futurs projets

Il y a trois habitudes que je veux garder au-delà de ce projet.

Éprouver les seuils en conditions réelles, pas sur le papier. Le calibrage SLO m'a montré qu'un chiffre recopié d'un workbook ne vaut rien tant qu'il n'a pas vu de trafic. Le réflexe vaut pour toute alerte que je poserai ensuite.

Documenter au fil de l'eau, pas à la fin. Les queries PromQL et les limites assumées, je les ai documentées trop tard. La documentation faite après coup est toujours moins juste que celle écrite au moment où on comprend.

Nommer franchement ce que je branche et ce que je maîtrise de bout en bout. Ce rapport en est un exemple. Distinguer les deux n'a rien d'un aveu de faiblesse, c'est ce qui rend une contribution défendable : un périmètre que je gonfle, je ne saurais pas le défendre à l'oral.